

JSON-RPC 2.0 Standard Compliance Assessment

VB.NET / Fortran Proof-of-Concept System

Revision 0, 6/21/26

Purpose and Scope

This document assesses the degree to which the proof-of-concept system — comprising the VB.NET JSON-RPC Client Program, the VB.NET JSON-RPC Client Library (DLL), the Fortran JSON-RPC Server Program, and the Fortran JSON-RPC Static Library — conforms to the requirements of the JSON-RPC 2.0 specification. The assessment is based on a review of the source code of all four components.

The JSON-RPC 2.0 specification uses normative language to distinguish between requirements of different strengths. A **MUST** requirement is mandatory: an implementation that does not satisfy it is non-conformant. A **SHOULD** requirement is a strong recommendation: an implementation may deviate from it for a documented reason, but conformant implementations are expected to satisfy it in the general case. **MAY** requirements describe optional features.

This document covers all **MUST** and **SHOULD** requirements of the JSON-RPC 2.0 specification. **MAY** (optional) requirements are not listed because their non-implementation carries no conformance implication; the optional "data" member in error objects is the only **MAY**-level feature present in the specification, and it is included in the table below because it is in fact implemented.

The intent of this table is to give a new reader an honest picture of where the system is fully conformant, where it departs from the specification in minor or constrained ways, and where a capability is absent. A proof-of-concept system is not expected to implement every aspect of a standard; the gaps identified here reflect deliberate simplifications rather than oversights.

Compliance Assessment

JSON-RPC 2.0 Requirement	Implementation Status
Request Object — MUST Requirements	
"jsonrpc" member must be present and must equal the string "2.0" exactly.	Fully Implemented. Client (StreamJsonRpc library) enforces this on all outbound requests. Server (Fortran static library) validates the field on every inbound message and returns -32600 if it is absent, not a string, or not equal to "2.0".
"method" member must be present and must be a non-empty string.	Fully Implemented. Client sends method names supplied by the calling application via StreamJsonRpc. Server validates that the field is present, is a JSON string type, and is not an empty string; each failure returns -32600.
Method names beginning with "rpc." are reserved for protocol-level use and must not be used by applications.	Fully Implemented. Server explicitly checks the first four characters of every method name and returns -32601 Method Not Found for any name beginning with "rpc.". Client does not use reserved names.
"params" member, if present, must be either a structured Array (positional) or Object (named); any other type is an error.	Fully Implemented. Client sends both positional arrays (for arithmetic calls) and named-parameter objects (for matrix and named-parameter calls) via StreamJsonRpc. Server validates the params field type and returns -32600 if it is present but is neither an Array nor an Object.
"id" member, if included, must be a String, Number, or NULL. Absent "id" identifies the message as a notification.	Partially Implemented. The server's raw validation layer (jsonrpc_validate_id_field) correctly accepts String, Number, and NULL values and rejects Booleans, Arrays, and Objects. However, the dispatch layer (jsonrpc_parse_message) only handles integer (and internally real-typed) IDs and returns -32600 for any ID that is not an integer, including the spec-permitted String type. The client (StreamJsonRpc) always sends sequential integer IDs and never sends String or NULL IDs, so this gap is not exercised in normal operation. It was decided NOT to

JSON-RPC 2.0 Requirement	Implementation Status
	implement the use of either a STRING or NULL for ID for simplicity.
Response Object — MUST Requirements	
Response must include a "jsonrpc" member with value "2.0".	Fully Implemented. The server adds "jsonrpc":"2.0" to every response and error response, including string-built fallback responses used when the json-fortran library is unavailable.
Response must include either a "result" member (on success) or an "error" member (on failure). The two members are mutually exclusive; a response must never contain both.	Fully Implemented. The server uses two separate response-building routines — jsonrpc_create_response (result path) and jsonrpc_create_error_response (error path) — which are called from mutually exclusive branches. No code path produces a response containing both members.
Response "id" must mirror the "id" from the corresponding request. If the request id cannot be determined (e.g. due to a parse error), the response "id" must be null.	Partially Implemented. Server correctly mirrors integer request IDs in all responses and uses null when the ID cannot be determined (parse error path, and pre-parse error responses). If a client sent a String ID, the server would reject it with -32600 and return id:null rather than mirroring the original String ID, which is not strictly conformant. In Fortran, nulls apply specifically to pointers and allocatable arrays to manage memory and association status, and it was decided not to implement this requirement for simplicity
Error Object — MUST Requirements	
Error object "code" member must be an integer.	Fully Implemented. All error codes defined in both the Fortran static library and the Fortran server are Fortran integer constants. The client (StreamJsonRpc / ErrorTestingModule) reads and compares error codes as integers.
Error object "message" member must be a short string describing the error.	Fully Implemented. Every error response includes a descriptive string message. The server applies JSON string escaping (escape_json_string /

JSON-RPC 2.0 Requirement	Implementation Status
	escape_simple) to all message strings before embedding them in responses.
Error object may include an optional "data" member containing additional error information.	Fully Implemented. SendErrorResponse accepts an optional error_data parameter; when supplied, it appends a "data" field to the error object. This is used for the oversized-request error (where data contains the actual vs. permitted byte counts) and is available for use by any handler.
Standard Error Codes — MUST Requirements	
-32700 Parse Error: must be returned when received JSON cannot be parsed.	Fully Implemented. Server returns -32700 when jsonrpc_extract_json fails to extract a valid JSON body, and when json-fortran's deserialise call reports a failure on the extracted content.
-32600 Invalid Request: must be returned when the JSON is valid but does not conform to the Request object structure.	Fully Implemented. Server returns -32600 for each of the following conditions: missing or non-string "jsonrpc" field, wrong version string, missing or non-string "method" field, empty method name, non-integer "id" field, and "params" field that is present but is neither Array nor Object. An empty batch array also returns -32600.
-32601 Method Not Found: must be returned when the named method does not exist on the server.	Fully Implemented. Server iterates the method registry and returns -32601 if no registered method matches the requested name. Reserved method names ("rpc.*") also return -32601 before the registry is consulted.
-32602 Invalid Params: must be returned when the method parameters are invalid.	Fully Implemented. Individual method handlers in the Fortran server return -32602 when the supplied parameters are the wrong count or wrong type. The error testing module on the client side verifies this by calling a known method with the wrong number of arguments.

JSON-RPC 2.0 Requirement	Implementation Status
-32603 Internal Error: must be returned for internal JSON-RPC processing errors.	Fully Implemented. Server returns -32603 for handler-level failures (e.g. division by zero, integer overflow when not caught by a more specific code) and for serialisation failures in the response-building path. The error testing module on the client side verifies this by invoking integer division by zero.
Application-defined server error codes should be in the reserved range -32000 to -32099.	Fully Implemented. The Fortran server defines five application-specific codes: -32000 (request exceeds maximum size), -32001 (matrix dimension mismatch), -32002 (invalid data type), -32098 (integer overflow), and -32099 (division by zero). All five are within the reserved range.
Notifications — MUST Requirements	
A Notification must not include an "id" member.	Fully Implemented. Client notifications sent via StreamJsonRpc's NotifyAsync / NotifyWithParameterObjectAsync correctly omit the "id" field. Server-initiated progress and status notifications are constructed as string literals without an "id" field.
Server must not send a Response object to a Notification.	Fully Implemented. Server sets an is_notification flag when the inbound message has no "id" field, and the dispatch path returns an empty string rather than a response object. The ProcessMessages loop suppresses the WriteFile call when the response string is empty.
Batch Requests — MUST Requirements	
An empty batch Array must be returned as an Invalid Request (-32600).	Fully Implemented. The server's jsonrpc_parse_batch routine checks for an empty array (num_requests == 0) and returns -32600 before processing any items.
If every item in a batch is a Notification, the server must not return any response.	Fully Implemented.

JSON-RPC 2.0 Requirement	Implementation Status
	The server tracks response_count; if it remains zero after processing all batch items (indicating all were notifications), it returns an empty string and the write is suppressed.
Batch response must be a JSON Array of the individual Response objects for the non-notification requests.	Fully Implemented (server side). Server assembles responses into a JSON Array and returns it as a single framed message. Order of responses within the array follows the order of requests in the batch.
Client should be capable of sending batch requests (multiple Request objects in a single JSON Array).	Not Implemented (client side). The VB.NET DLL (RPCOperations module) provides no batch-sending function. The "batch test" in the client application invokes individual RPC calls sequentially, not packaged into a JSON Array. The server fully supports receiving and processing batch requests, but the client never sends them. See below for more details.
SHOULD Requirements (Recommended Behaviors)	
The "id" member should not be Null in a Request. Null is discouraged in requests because it is reserved to indicate an unknown ID in error responses, and because JSON-RPC 1.0 used Null for Notifications.	Fully Implemented. The client (StreamJsonRpc) never sends Null as a request ID; it uses sequential integers starting from 1. If a message arrives with id:null, the server correctly treats it as a Notification (no response sent), which is the specified fallback behaviour.
The "id" member should not use fractional numbers because many decimal fractions cannot be represented exactly in binary floating-point, which may cause ID mismatch between client and server.	Partially Implemented. The client always sends integer IDs — fractional IDs are never generated. However, the server dispatch code (jsonrpc_parse_message) accepts json_real in addition to json_integer when checking the ID field type, due to a json-fortran library quirk where some integer literals are classified internally as real values. In principle this means a fractional ID arriving from a third-party client would be accepted rather than rejected, contrary to the spec's guidance.

Summary of Findings

Fully Implemented (21 of 24 requirements)

The large majority of JSON-RPC 2.0 requirements are fully satisfied. All five standard error codes are correctly defined, triggered, and tested. The request validation pipeline on the server side checks every mandatory field (jsonrpc version, method type and value, params structure, reserved method names). Notification handling is correct on both sides: the client sends notifications without an ID field, and the server suppresses responses to notifications. Batch request processing on the server is complete, including the edge cases of empty arrays and all-notification batches. The server's application-specific error codes all fall within the reserved range.

Partially Implemented (2 of 24 requirements)

Two requirements are satisfied for the integer-ID subset that the system actually uses, but would not be satisfied for the full range of ID types the specification permits. The specification allows a request ID to be a String, Number, or NULL, but the server's dispatch layer only accepts integer (number without fraction) IDs; a client sending a String ID would receive a -32600 Invalid Request error rather than having the call dispatched. The consequence is that the response ID cannot mirror a String request ID, which is the second partially-implemented item. In practice this gap has no effect because the VB.NET client library always sends integer IDs. A second minor gap is that the server's ID-type check accepts json_real in addition to json_integer as an accommodation for a json-fortran library behavior, which would permit a fractional ID to pass validation despite the specification's guidance to avoid them.

Not Implemented (1 of 24 requirements)

The VB.NET client does not send batch requests that are compliant with the standard. The JSON-RPC 2.0 specification envisions that both sides of a connection may package multiple Request objects into a single JSON Array for transmission in one framed message. The server's batch-handling code (jsonrpc_parse_batch) is fully implemented and would correctly process any batch it receives. However, the VB.NET DLL provides no function for constructing or sending a batch array, and the client application's "batch test" exercises sequential single calls rather than a true batch. This is a limitation of the current version of the Microsoft StreamJsonRpc library, see below.

Batch requests – not properly implemented

There is a button on the VB.NET client form for testing the sending of a batch request to the Fortran server. The VB client and Fortran server programs *appear* to properly implement the sending of a batch request and receiving the results. However, the assessment of this standard requirement reveals it is “cheating”. The analysis of the cause of the non-compliance and subsequent architecture changes to fully implement batch requests was deemed to be something left for the Revision 1 of these files, as changes would be required for all 4 programs.

The main problem lies with the use of StreamJsonRpc, as follows:

The core architectural obstacle: StreamJsonRpc owns the pipe stream

This is the critical finding. Inside ConnectionModule.InitializeJsonRpc:

```
Dim messageHandler = New HeaderDelimitedMessageHandler(pipeClient, pipeClient, formatter)
Dim rpc = New JsonRpc(messageHandler, rpcTarget)
rpc.StartListening()
```

When StartListening() is called, StreamJsonRpc starts a continuous read loop on the NamedPipeClientStream. From that point, the stream is exclusively owned by StreamJsonRpc. Two consequences follow:

1. Writing a raw JSON array directly to pipeClient is not safe. StreamJsonRpc's write path goes through HeaderDelimitedMessageHandler, which adds the Content-Length header. Bypassing this and writing raw bytes to pipeClient directly would produce a malformed frame the Fortran server cannot parse, and any response bytes received back would be consumed by StreamJsonRpc's read loop, not by the batch caller.
2. StreamJsonRpc v2.24.84 provides no public batch-sending API. There is no InvokeBatchAsync or equivalent. Every call to InvokeAsync or NotifyAsync produces exactly one JSON object, never a JSON array. There is therefore no way to send a compliant JSON-RPC 2.0 batch through the existing JsonRpc instance while StreamJsonRpc's session is active.

Various methods have been proposed for correction, but in the interest of time, this exception is noted as for resolution in a future revision of the programs.